

AI WARDROBE STYLIST

Project Documentation

OOTD AI Fashion Stylist — Functionality & Intelligence Upgrade

Group Project

Prepared by: Khensani Ntombela and project team

Project Team:

Khensani Ntombela

Baphakamise Ncayo

Samkelo Mngadi

Elzel De Beer

Refilwe Molelu

2026

1. Executive Summary

AI Wardrobe Stylist is a generative-AI fashion styling and digital wardrobe application designed to help users make personalised outfit decisions using the clothes and accessories they already own. The project evolved from a partially completed prototype into a more ambitious product concept focused on contextual reasoning, wardrobe-aware recommendations, conversation, inspiration discovery and personal outfit organisation.

The central idea is that an AI stylist should understand what a user is asking for in natural language, interpret the occasion and desired aesthetic, consider practical context such as weather when relevant, identify suitable items in the user's wardrobe, and explain why the resulting outfit works. The application also connects styling conversations with saved outfits, wear history and inspiration.

The project demonstrates a practical application of generative AI and prompt engineering to an everyday problem while placing emphasis on user experience, personalisation, persistence, responsible handling of external services and meaningful AI-driven functionality.

2. Project Background

The project began with an existing OOTD (Outfit of the Day) AI Fashion Stylist web application that contained a number of interface sections and partially implemented functionality. Rather than rebuilding the product from the beginning, the development brief required the existing visual identity, layout, wardrobe data and overall concept to be preserved while functionality and intelligence were significantly improved.

The resulting concept combines an AI stylist, digital wardrobe, inspiration discovery, saved outfits, boards, wear logs, chat history, profile and settings into one connected experience.

3. Problem Statement

Choosing an outfit can involve more than selecting individual clothing items. Users may need to consider the occasion, dress code, weather, comfort, personal style, colour combinations, formality and the clothing they already own. A conventional fashion recommendation system may also recommend generic or unrelated items without understanding the user's actual wardrobe.

The project therefore addresses the need for a more personalised styling experience in which users can describe what they want naturally and receive recommendations grounded in their existing wardrobe and the context of the request.

4. Aim and Objectives

The overall aim is to create a functional, intelligent and user-friendly AI fashion styling application rather than a static visual prototype.

- Enable natural-language interaction with an AI fashion stylist.
- Interpret occasion, formality, mood, aesthetic, weather, practicality and other contextual cues.
- Prioritise the user's actual wardrobe when generating outfit recommendations.
- Evaluate compatibility between multiple wardrobe items before presenting an outfit.
- Use inspiration to inform styling without replacing the user's wardrobe as the source of truth.

- Preserve conversations, outfits, wardrobe data and relevant user settings during use.
- Connect chats, saved outfits, boards and wear logs into a coherent user experience.
- Provide useful actions such as saving, modifying, wearing and organising recommended outfits.
- Handle missing information, API failures and unavailable wardrobe matches transparently.
- Maintain a consistent fashion/editorial visual identity across desktop and mobile experiences.

5. Target Users

The primary users are people who want help planning outfits and discovering ways to style clothing they already own. The application is particularly relevant to users who value personalisation, fashion inspiration, wardrobe organisation and conversational AI assistance.

6. Core Features and Functionality

Feature	Purpose
AI Stylist	Conversational styling assistant that interprets natural-language requests and provides contextual recommendations.
Digital Wardrobe	A personal catalogue of clothing, shoes, bags, jewellery and accessories that serves as the source of truth for styling.
Historic Chats	A dedicated space for retaining styling conversations so users can reopen, continue, rename or delete conversations.
Pinterest / Inspiration	An inspiration workflow that interprets a requested aesthetic, finds relevant style inspiration and relates the style to the user's wardrobe.
Saved Outfits	A space for retaining useful outfit combinations and treating them as reusable personal inspiration.
Boards	Pinterest-style organisation of saved outfits into themes such as Workwear, Date Night, Minimalist or Wedding Guest.
Wear Logs	A record of outfits that have been worn, with the ability to connect an entry to its originating styling conversation where applicable.
Profile	Personal user information kept separate from application preferences.
Settings	Application controls including language, appearance and curated accent-colour options.
Responsive Experience	The application is intended to remain usable across desktop, laptop, tablet and mobile layouts.

7. AI Intelligence and Prompt Engineering

Generative AI is the core intelligence layer of the application. Prompt engineering was used to guide the system away from rigid, form-based questioning and toward a more natural conversation. The intended behaviour is for the AI to infer relevant context from ordinary language instead of requiring the user to explicitly provide every category.

7.1 Natural-language understanding

Examples of natural requests include: “I am attending an important meeting for work”, “I have a date tonight”, “I need something elegant for dinner”, “I want something comfortable but still put together”, or “I want a Parisian look using what I already own.”

From such requests, the stylist is designed to infer relevant factors such as:

- occasion
- environment
- formality and dress code
- mood and aesthetic
- weather requirements
- practicality
- colour compatibility
- garment compatibility
- silhouette
- season
- time of day
- stated user preferences

7.2 Intelligent follow-up questions

The AI should ask for additional information only when it is necessary to produce a meaningful recommendation. For example, a work-meeting request may require the date and location if weather materially affects the outfit. This approach avoids unnecessary questioning while preserving the information needed for contextual recommendations.

7.3 Conversational memory

Within a styling conversation, the AI is expected to retain context. A user should be able to say “make it more feminine”, “change the shoes”, or “keep the blazer but change everything else” and have the stylist modify the existing recommendation rather than treating each message as a completely new request.

8. Wardrobe-First Recommendation Approach

A defining principle of the application is that the user's actual wardrobe is the source of truth. Recommendations should not simply select arbitrary items because they exist in the database. Instead, the AI should evaluate wardrobe items against the user's requested occasion, aesthetic and context.

Where available, wardrobe attributes may include:

- item name
- category and subcategory
- colour
- brand
- formality
- occasion
- style
- season

- material
- silhouette
- compatibility
- user preferences
- wear history

This approach also supports wardrobe-gap intelligence. If the wardrobe cannot produce a complete outfit, the system should identify what is missing rather than inventing an item or returning an arbitrary combination.

9. Outfit Compatibility Engine

The project treats an outfit as a complete combination rather than a collection of unrelated items. Before presenting a recommendation, the intended reasoning considers:

- colour harmony
- formality compatibility
- occasion compatibility
- silhouette compatibility
- style compatibility
- weather suitability
- practicality
- layering
- footwear compatibility
- accessory balance

This is intended to reduce recommendations that technically contain available wardrobe items but do not make sense together as a complete look.

10. Weather-Aware Styling

Weather is treated as contextual information rather than a decorative feature. When a request depends on weather, the application is designed to determine the relevant date and location and, where an appropriate service is available, retrieve relevant weather information.

- temperature
- rain
- wind
- humidity
- extreme heat or cold
- changes throughout the day

If weather information cannot be retrieved, the application should communicate this clearly instead of pretending that weather data was checked.

11. Pinterest-Inspired Styling and Discovery

The inspiration component is intended to move beyond simply displaying fashion images. The workflow begins with the user's request, interprets the requested aesthetic, identifies relevant style inspiration, analyses characteristics of that inspiration, and then matches those characteristics against the user's actual wardrobe.

The intended flow is:

1. User describes a desired style or occasion.
2. AI interprets the request into style characteristics.
3. Relevant inspiration is identified.
4. The inspiration is considered in terms of palette, silhouette, layering, garments, accessories, footwear and styling details.
5. The style characteristics are compared against the user's wardrobe.
6. The application explains why selected pieces fit the requested aesthetic.
7. Missing item types can be identified when the wardrobe cannot fully reproduce the look.

12. Persistence and Data Integrity

The project specification places strong emphasis on preserving user-created information during navigation and refreshes. The intended persistent application state includes wardrobe items, saved outfits, boards, wear logs, conversations, profile information and settings.

For a front-end-only environment, browser-side persistence such as `localStorage` is an appropriate approach where suitable. A backend should only be introduced when genuinely required.

Data integrity is also important when records reference one another. For example, deleting a wardrobe item should not break an older saved outfit or wear log; the application should handle missing references gracefully.

13. Key User Flows

13.1 Natural-language styling flow

USER REQUEST → AI INTERPRETS INTENT → CONTEXT CHECK → WARDROBE MATCH → COMPATIBILITY CHECK → OUTFIT + EXPLANATION → USER ACTION

13.2 Inspiration-to-outfit flow

DISCOVER INSPIRATION → UNDERSTAND STYLE → MATCH AGAINST WARDROBE → IDENTIFY OWNED ITEMS → IDENTIFY GAPS → SAVE → ADD TO BOARD → CREATE OUTFIT

13.3 Conversation flow

START CHAT → STYLE REQUEST → FOLLOW-UP IF NEEDED → RECOMMENDATION → MODIFY → SAVE / WEAR / BOARD → CHAT HISTORY

14. User-Controlled Actions

- Save Outfit
- Wear Today
- Add to Board
- View Inspiration
- Try Another
- Modify Outfit
- Start New Chat
- Rename Chat
- Continue Chat
- Delete Chat

The design principle is that an action label should correspond to a real operation. A Save action should save, Delete should delete, Rename should rename, and View Chat should restore the relevant conversation.

15. User Experience and Design

The project was instructed to preserve its existing fashion/editorial identity rather than introducing an unrelated redesign. The intended visual language includes soft neutral colours, cream, alabaster, blush, sage, terracotta and charcoal, with rounded cards, consistent spacing and a responsive layout.

The interface should remain coherent across the sidebar, headers, cards, forms, modals, chat, buttons, inputs, notifications and image containers.

16. Accessibility and Usability

- meaningful labels for buttons
- tooltips and accessible names for icon-only controls
- keyboard interaction
- visible focus states
- sensible form validation
- clear close/cancel controls for modals
- usable layouts on smaller screens

17. API Integration and Security

The project brief identifies API security as an important consideration. Sensitive API keys should not be hard-coded into production client-side code. Where external services require credentials, secure server-side configuration or environment variables should be used. For a front-end-only prototype, integrations should be structured so that secure connectivity can be introduced later without rewriting the entire application.

18. Error Handling

External and API-dependent features should fail transparently. If weather or inspiration data cannot be retrieved, the application should explain the issue, provide a retry option where appropriate, and avoid inventing information.

A key quality rule is the removal of arbitrary fallback outfits. If no valid combination exists, the system should explain the limitation and identify the missing item type or other reason for the incomplete recommendation.

19. Testing and Quality Assurance

The project brief requires a full functionality audit before final delivery. Testing areas include:

Area	Examples of checks
Navigation	Sidebar, desktop/mobile navigation, collapse/expand
Wardrobe	Add, upload, edit, rename, delete, search, categories
AI Stylist	Natural-language requests, follow-ups, modifications, contextual memory
Weather	Relevant requests, successful retrieval, failure handling
Inspiration	Search, style interpretation, wardrobe matching, saving
Chats	Create, save, rename, continue, delete, restore
Wear Logs	Add, cancel, view modes, associated chat
Saved Outfits	Save, delete, create, cancel, boards
Boards	Create, rename, delete, open, add/remove/move outfits
Profile	Edit and save personal details
Settings	Language, appearance, accent colour, persistence
Responsive UX	Desktop, laptop, tablet and mobile usability
Data integrity	Deleted wardrobe items do not break existing records

20. Development Approach

The development instruction explicitly followed an enhancement-and-refactoring approach rather than a greenfield rebuild. The first priority was to inspect the existing architecture and identify broken state flows, unused state, dead functions, fallback logic, missing handlers, incomplete modals and API integration issues.

- Preserve the existing application and visual identity.
- Identify disconnected or competing implementations.
- Refactor broken state flows before adding new functionality.
- Remove obsolete or duplicate logic when replacing functionality.
- Connect related features instead of treating each page as an isolated component.
- Test user-facing interactions before final delivery.

21. Tools and Technologies

Tool / Technology	Role in the project
Generative AI	Core intelligence and conversational styling capability
Prompt Engineering	Guiding AI behaviour, context interpretation and response quality
Gemini ChatGPT	AI development environment and project interaction Generative-AI assistance during project development and refinement
Web Application Interface	User-facing wardrobe, styling, inspiration and organisation experience
Browser-side persistence	Suitable mechanism for retaining front-end application state where required

The exact implementation technologies should be listed in the final technical appendix according to the actual project codebase. This documentation intentionally avoids claiming a framework or backend technology that was not specified in the project brief.

22. Key Development Challenges

- Moving from a visual prototype to genuinely functional interactions.
- Maintaining state across multiple application sections.
- Making AI responses context-aware rather than dependent on rigid question flows.
- Ensuring recommendations are grounded in the user's actual wardrobe.
- Connecting inspiration, wardrobe data and AI reasoning.
- Avoiding misleading fallback recommendations when no suitable outfit exists.
- Maintaining a coherent experience while adding a large number of interconnected features.
- Balancing functionality, responsiveness, accessibility and visual consistency.

23. Learning Outcomes

- Practical application of generative AI to a real user problem.
- Prompt engineering for contextual and conversational AI behaviour.
- Thinking about AI outputs as product experiences rather than isolated responses.
- Designing state and data relationships across multiple application features.
- Understanding the importance of error handling and transparent AI limitations.
- Considering accessibility, responsive design and usability alongside functionality.
- Working collaboratively on a product concept and documenting the development process.

24. Future Improvements

- Secure production-grade backend/API architecture where needed.
- More sophisticated wardrobe attribute extraction and compatibility scoring.
- Improved visual analysis of uploaded clothing and inspiration images.
- Broader multilingual coverage and more complete interface translations.
- More advanced personalisation based on long-term user preferences and wear history.
- Enhanced recommendation explanations and outfit comparison tools.

- Additional integrations for weather, shopping or calendar-based occasion planning, subject to privacy and security considerations.

25. Final Quality Standard

The intended final experience is a genuine AI-powered personal fashion stylist and digital wardrobe platform rather than a static fashion dashboard. The central experience is:



26. Project Links

AI Wardrobe Stylist (Gemini):

<https://gemini.google.com/share/bfb8b44fc16a?skid=f0551d75-2f91-4471-bc0f-d478a1a0c7d7>

Portfolio:

<https://khensanintombela.github.io/My-Portfolio/>

27. Documentation Note

This document is based on the original functionality and intelligence-upgrade brief supplied for the project. It describes the intended product behaviour, design principles, development approach and quality requirements. Before formal submission, the team should update the testing section with actual test results and add screenshots of the final application to demonstrate the implemented features. Any feature that was planned but not ultimately implemented should be marked as a future improvement rather than presented as completed functionality.